

Penjelasan Kode: Game Explore

Bahan video penjelasan kode (bukan demo aplikasi). Arsitektur MVVM: data (Retrofit, repository) -> ViewModel -> Compose (state-driven UI).

0. Pembuka

Aplikasi katalog video game Kotlin + Jetpack Compose, data real-time dari REST API RAWG. Struktur paket: data/model (Game.kt), data/remote (RawgApiService, RetrofitClient), data/repository (GameRepository), ui/navigation (NavGraph), ui/screen (HomeScreen, GameDetailScreen), ui/theme (Color, Type, Theme), ui/viewmodel (GameViewModel), util (Constants, Format).

1. util/Constants.kt — Titik Masuk API

object Constants berisi BASE_URL 'https://api.rawg.io/api/' dan API_KEY, dipakai semua layer, mudah diganti di satu tempat. Key hardcoded agar aplikasi langsung jalan setelah clone; untuk produksi sebaiknya dipindah ke local.properties + BuildConfig.

2. data/model/Game.kt — Data Class & Null Safety

GameResponse membungkus results; Game memetakan id, name, rating, released, background_image, description, description_raw, genres; Genre (id, name). @SerializedName mencocokkan JSON snake_case ke properti Kotlin (camelCase). Field nullable (String?, Double?, List<Genre>?) adalah null safety: RAWG tidak menjamin semua field ada (released bisa null -> tampil 'TBA'). Deskripsi lengkap hanya ada di endpoint detail (description_raw), sebab itu list tidak menampilkan deskripsi.

3. data/remote — Retrofit (Networking)

RetrofitClient: object singleton, property by lazy, builder baseUrl + GsonConverterFactory, build sekali lalu dipakai bersama. RawgApiService: interface dua endpoint suspend — @GET('games') getGames(@Query key, @Query search default null) dan @GET('games/{id}') getGameDetail(@Path id, @Query key). Fungsi suspend berjalan di coroutine sehingga UI tidak freeze; default parameter membuat endpoint sama dipakai untuk daftar awal maupun search; @Path mengisi {id} sehingga URL jadi games/3498?key=...

4. data/repository/GameRepository.kt — Lapisan Repository

Satu-satunya pintu keluar data: ViewModel tidak pernah memanggil Retrofit langsung. getGames(search) mengembalikan response.results ?: emptyList() (operator elvis menangani null). Mengganti API ke RAWG cukup di file ini; API key disisipkan di sini, tidak bocor ke UI.

5. ui/viewmodel/GameViewModel.kt — Inti MVVM & Search

(a) Sealed interface HomeUiState dan DetailUiState: hanya bisa Loading, Success, atau Error — state selalu valid, when exhaustive dipaksa kompiler, recomposition efisien. (b) var homeUiState by mutableStateOf(...) dengan private set: Compose mengamati, layar hanya membaca, perubahan lewat fungsi ViewModel (encapsulation). (c) Search: updateSearchQuery(query) — searchQuery langsung diperbarui (kotak responsif), searchJob?.cancel() membatalkan timer sebelumnya, lalu delay(300) baru fetchGames — ini debounce, mencegah request berlebihan (RAWG gratis +-20 request/menit); requestId (++homeRequestId) hanya membiarkan respons terakhir menulis state — anti race condition; keyword di-trim, kosong -> null; try-catch -> HomeUiState.Error + tombol Coba Lagi, bukan crash. (d) resetDetailState() dipanggil sebelum navigasi agar detail tidak sempit

menampilkan data game sebelumnya (stale state).

6. ui/navigation/NavGraph.kt — Dua Layar + Argument

sealed class Screen: Home("home") dan Detail("detail/{gameId}", createRoute dengan string template). NavHost startDestination home; route detail punya navArgument Int; gameId dibaca dari backStackEntry; satu instance GameViewModel dipakai bersama kedua layar sehingga state search tetap tersimpan saat kembali; resetDetailState() sebelum navigate; popBackStack untuk tombol kembali.

7. ui/screen/HomeScreen.kt — Lazy Layout & Kartu Game

Scaffold + TopAppBar + SearchField; konten dipilih when(uiState): Loading -> CircularProgressIndicator; Success kosong -> EmptyState('Game tidak ditemukan'); Success berisi -> LazyVerticalGrid GridCells.Fixed(2), Arrangement.spacedBy(12), items key={it.id} — grid 2 kolom (pilih salah satu lazy layout sesuai spek), item dibuat saat tampil, key unik untuk akurasi scroll/perubahan; Error -> EmptyState + tombol Coba Lagi memanggil fetchGames(searchQuery). Unidirectional data flow: event naik ke ViewModel, state turun. Kartu: AsyncImage (Coil) background_image ContentScale.Crop + overlay gradasi Brush.verticalGradient, judul, formatRating, tanggal rilis ISO 8601 (atau TBA).

8. ui/screen/GameDetailScreen.kt — Detail & Lifecycle

LaunchedEffect(gameId) menjalankan fetchGameDetail tepat satu kali saat layar muncul, dibatalkan saat layar ditutup. when detailUiState: Loading spinner; Success -> gambar banner, judul headlineSmall, kartu rating (formatRating + ratingStars), kartu tanggal rilis, chips genre (FlowRow + SuggestionChip), deskripsi bersih dari HTML via Html.fromHtml(...) — descriptionRaw diprioritaskan, fallback description, lalu teks default; Error -> tombol retry fetchGameDetail(gameId).

9. util/Format.kt — Fungsi Top-Level

formatRating: Double? -> '4.5' atau 'N/A', memakai Locale.ROOT agar selalu titik bukan koma (tidak ikut locale perangkat). ratingStars: membulatkan ke 0–5 dengan coerceIn, membangun string '*****' dengan repeat. Fungsi top-level = berdiri sendiri, dipakai dua layar tanpa duplikasi (DRY).

10. ui/theme — Material Design 3 (Spek b)

GameExploreTheme menggabungkan colorScheme (Dark/Light) + typography kustom ke MaterialTheme. Color.kt: palet gaming ungu #8B5CF6, aksen cyan #22D3EE, pink #FF6B9D, background near-black; dynamicColor dimatikan agar identitas konsisten. Type.kt: typography lengkap (headline/title/body/label), judul tebal, bodyMedium lineHeight nyaman untuk deskripsi panjang. Mode gelap-terang mengikuti sistem (isSystemInDarkTheme).

11. MainActivity.kt — Jembatan Android & Compose

ComponentActivity tunggal. enableEdgeToEdge() membuat konten menembus status bar. setContent { GameExploreTheme { NavGraph() } } adalah titik masuk Compose — UI dideklarasikan sebagai fungsi Composable, bukan XML.

12. Alur Data End-to-End

Ketikan di SearchField -> onQueryChange -> updateSearchQuery (searchQuery tampil seketika, timer lama dibatalkan) -> 300 ms debounce -> fetchGames -> repository -> Retrofit GET /games?search=...&key=... (coroutine) -> Gson jadi List<Game> -> requestId valid -> state Loading -> Success(games) -> Compose recomposition hanya grid (state-driven). Gagal -> Error; kosong -> 'Game tidak ditemukan'. Klik kartu -> navigate detail/{id} -> LaunchedEffect -> detail -> description_raw dibersihkan HTML.

13. Pemetaan ke Spek Responsi

a. Kotlin (data class, null safety, default param): Game.kt, RawgApiService, Format.kt. b. Compose + M3 + theme/typography: ui/screen, ui/theme. c. LazyVerticalGrid: HomeScreen. d. State-driven + search: GameViewModel (sealed state, debounce 300 ms, requestId). e. Networking RAWG (nama, rating, tanggal ISO 8601, deskripsi): RetrofitClient, RawgApiService, Game.kt. f. MVVM (Retrofit, repository): data -> GameViewModel -> ui. g. Dua layar (Home + Detail): NavGraph. h. README + video: repository root.

14. Penutup

MVVM memisahkan data, logika, tampilan; coroutine + suspend menjaga UI responsif; sealed interface membuat state selalu valid; debounce + requestId membuat search cepat dan aman dari race; seluruh tampilan Material Design 3 dengan theme kustom.